

به نام خدا



درس سیستم عامل
نیم سال اول ۰۵-۰۴
استاد: دکتر اسدی

دانشکده مهندسی کامپیوتر

تمرین سری هفتم

- پرسش‌های خود را در سامانه CW و تالار مربوط به تمرین مطرح نمایید.
- پاسخ سوالات را تایپ نمایید.
- اسکرین‌شات‌ها، عکس‌ها، فایل‌های مربوط به سوال عملی، گزارش تمرینات عملی و PDF قسمت تئوری را در پوشه با نام به فرمت HWNUM_StudentID1_StudentID2 ذخیره نمایید. سپس آن را zip نمایید و در صفحه درس بارگذاری نمایید. به عنوان مثال یک فایل بارگذاری شده قابل قبول باید دارای فرمت HW1_400123456_403123456.zip باشد.
- هر دانشجو می‌تواند حداکثر سه تمرین را با دو روز تأخیر بدون کاهش نمره ارسال نماید.
- تمرینات عملی و تئوری به صورت گروه‌های دو نفر تحویل داده شود.
- هر دو عضو گروه موظف هستند تمرینات خود را بارگذاری کنند.
- عواقب عدم تطابق بین پاسخ دو عضو گروه برعهده خودشان است.
- تحویل تمرین به صورت انگلیسی مجاز نیست. در صورت تحویل تمرین به صورت انگلیسی (حتی بخشی از تمرین) نمره تمرین موردنظر صفر در نظر گرفته می‌شود.
- در صورت مشاهده تقلب برای بار اول نمره هر دو طرف صفر می‌شود. در صورت تکرار نمره کل تمرینات صفر خواهد شد.
- استفاده از ابزارهایی مانند ChatGPT به منظور ابزار کمک آموزشی مجاز است به شرط آن که به خروجی آن اکتفا نشود.
- توجه شود که پروژه نهایی درس در گروه‌های چهار نفر تحویل گرفته می‌شود.
- سوالات با عنوان اختیاری نمره‌ای ندارند اما جواب دادن به آن‌ها کمک به سزایی در یادگیری درس می‌کند.
- هیچ تمرینی خارج از CW تصحیح نخواهد شد. لذا توصیه می‌شود حداقل یک الی دو ساعت قبل از زمان پایان تمرین برای بارگذاری تمرین اقدام شود.

تمارین تئوری

۱. به سوالات زیر پاسخ دهید.

(آ) در یک cache، بیت‌های tag از سمت چپ آدرس (بیت‌های با ارزش بیشتر) گرفته می‌شوند، در حالی که بیت‌های index و offset از بیت‌های کم‌اهمیت‌تر انتخاب می‌شوند. چرا منطقی نیست که cache index را از بیت‌های با ارزش بیشتر بگیریم؟ توضیح دهید.

(ب) شما یک کامپیوتر خریده‌اید که دارای فضای آدرس ۶۴ بیتی، مقدار 32 GiB حافظه اصلی، یک پردازنده گرافیکی NVIDIA GeForce RTX 3090 Ti و یک SSD با حجم 16 TiB است. تصمیم گرفته‌اید یک سیستم عامل ۶۴ بیتی برای آن بنویسید. برای کاهش تأخیر در ترجمه‌ی آدرس‌ها، می‌خواهید از جدول صفحه یک سطحی استفاده کنید، اما همچنان از چیدمان استاندارد حافظه بهره می‌برید (یعنی stack از بالای فضای آدرس به سمت پایین رشد می‌کند و heap رو به بالا رشد می‌کند). فرض کنید هر page اندازه‌ی 4 KiB دارد. آیا این روش قابل اجرا است یا خیر؟ توضیح دهید.

(ج) فرض کنید الگوی دسترسی برنامه‌ی شما به صفحات به صورت زیر است و تعداد فریم‌های cache برابر ۳ است.

A B C D A B D C B A

(آ) چند miss با سیاست FIFO خواهید داشت؟

(ب) چند miss با سیاست LRU خواهید داشت؟

۲. فرض کنید می‌خواهید یک طرح حافظه با جدول صفحه یک سطحی بسازید. هر دسترسی به حافظه اصلی 50 ns طول می‌کشد و هر دسترسی به TLB، 10 ns است.

(الف) فرض کنید هیچ خطای صفحه‌ای^۱ پیش نمی‌آید (یعنی تمام فضای حافظه مجازی در حافظه حضور دارد). برای داشتن AMAT برابر 61 ns، TLB باید چه ضریب برخوردی^۲ داشته باشد؟

(ب) اگر ضریب برخورد TLB برابر 50% باشد، AMAT در این حالت نسبت به نبودن TLB چگونه است؟

(ج) برای بهبود سیستم، یک صفحه‌بندی دو سطحی و یک Cache اضافه می‌کنید. Cache دارای ضریب برخورد 90% است و زمان جست‌وجوی آن 20 ns است. همچنین ضریب برخورد TLB برابر 95% است. میانگین زمان خواندن یک مکان از حافظه چقدر می‌شود؟

۳. (آ) ماشینی را با حافظه فیزیکی 8 GB، اندازه صفحه 8 KB و اندازه مدخل جدول صفحه^۳ برابر با 4 bytes در نظر بگیرید. اگر هر جدول صفحه بتواند در یک تک صفحه جای بگیرد، چند سطح از جداول صفحه برای نگاشت یک فضای آدرس مجازی 46-bit مورد نیاز است؟

(ب) فیلدهای یک مدخل جدول صفحه را در طرح خود فهرست کنید.

(ج) بدون حافظه نهان یا TLB، برای خواندن یا نوشتن یک کلمه 32-bit، چند عملیات حافظه مورد نیاز است؟

(د) با وجود یک TLB، این تعداد به چند عملیات حافظه کاهش می‌یابد؟ سناریوی بهترین حالت و سناریوی بدترین را بررسی نمایید.

(ه) ماشینی را با اندازه صفحه 1024 bytes در نظر بگیرید. مقدار 8KB حافظه فیزیکی و 8KB حافظه مجازی وجود دارد. TLB یک حافظه نهان کاملاً انجمنی^۴ با فضا برای 4 ورودی است که در حال حاضر خالی است. فرض کنید که شماره صفحه فیزیکی (PPN) همیشه یکی بیشتر از شماره صفحه مجازی (VPN) است. دنباله‌ی زیر آدرس دسترسی‌های به حافظه توسط یک برنامه است (به ترتیب از چپ به راست):

¹Page Fault

²Hit Rate

³Page Table Entry

⁴Fully Associative

0x294, 0xA76, 0x5A4, 0x923, 0xCFF, 0xA12, 0xF9F, 0x392, 0x341

در جدول زیر وضعیت فعلی جدول صفحه آمده است:

Physical Page Number	Valid Bit
NULL	۰
۲	۱
NULL	۰
۴	۰
۵	۰
۶	۱
۷	۱
NULL	۰

چه تعداد برخورد TLB و خطای صفحه وجود دارد؟ محتویات TLB در پایان این دنباله چیست؟

۴. به سوالات زیر همراه با استدلال پاسخ دهید.

(آ) آیا صفحه‌بندی باعث پراکندگی خارجی^۵ می‌شود؟

(ب) آیا ریس‌ها می‌توانند یک فضای آدرس مشترک^۶ داشته باشند؟

(ج) آیا الگوریتم LRU می‌تواند پدیده‌ی آنومالی بلدی^۷ را نشان دهد؟

(د) فرض کنید سیستمی با یک فرایند وجود دارد که مجموعه‌ی کاری^۸ آن در حافظه اصلی جا می‌شود. سیستم با صفحه‌بندی درخواستی^۹ و الگوریتم جایگزینی صفحه‌ای به جز LRU (مثلاً CLOCK) کار می‌کند. ناگهان این فرایند N ریس‌ه یکسان ایجاد می‌کند که هرکدام به همان صفحات موجود در مجموعه‌ی کاری دسترسی دارند. آیا با وجود این که مجموعه‌ی کاری کل تمام نخ‌ها برابر با مجموعه‌ی کاری فرایند تک‌ریسه است، نرخ خطای صفحه کل ممکن است پس از ایجاد نخ‌ها افزایش یابد؟

۵. یک فضای آدرس مجازی ۳۲ بیتی را در نظر بگیرید که اندازه هر صفحه برابر ۴ کیلوبایت است. سیستم از طرح صفحه‌بندی دو سطحی استفاده می‌کند.

• جدول صفحه خارجی با استفاده از بیت‌های بالایی آدرس اندیس گذاری می‌شود.

• جدول صفحه داخلی با استفاده از بیت‌های میانی اندیس گذاری می‌شود.

• هر ورودی جدول صفحه (PTE) برابر ۴ بایت است.

• سیستم طوری طراحی شده است که جدول صفحه خارجی دقیقاً در یک صفحه حافظه قرار بگیرد.

(آ) تقسیم‌بندی بیت‌های آدرس مجازی را مشخص کنید (مثلاً X بیت برای جدول خارجی Y بیت برای جدول داخلی، Z بیت برای Offset)

(ب) اگر یک فرایند دقیقاً ۱۲ مگابایت حافظه مجازی را از آدرس 0x00000000 به صورت پیوسته استفاده کند، میزان حافظه فیزیکی مورد نیاز برای تمام جدول‌های صفحه (خارجی و داخلی) را محاسبه کنید.

(ج) این مقدار را با اندازه یک جدول صفحه یک سطحی که تمام فضای ۳۲ بیتی را پوشش دهد مقایسه کنید.

(د) برای آدرس مجازی 0x00403004

⁵External Fragmentation

⁶Address Space

⁷Belady's Anomaly

⁸Working Set

⁹Demand Paging

- اندیس جدول صفحه خارجی را به دست آورید.
- اندیس جدول صفحه داخلی را به دست آورید.

۶. یک شرکت در حوزه داده‌های بزرگ شما را استخدام کرده است تا به طراحی سیستم حافظه جدید آن‌ها برای یک سیستم بایت-آدرس‌پذیر کمک کنید. فرض کنید فضای آدرس حافظه مجازی و فیزیکی ۳۲ بیتی با اندازه صفحه 4KB باشد.

(آ) فرض کنید می‌دانید که در هر لحظه تنها ۴ فرآیند در حال اجرا خواهند بود، هر کدام با اندازه مجموعه مقیم^{۱۰} برابر با 512MB و اندازه مجموعه کاری^{۱۱} برابر با 256KB است. حداقل تعداد مدخل‌های TLB که سیستم شما برای نگاشت/کش کردن اندازه مجموعه کاری یک فرآیند نیاز دارد چقدر است؟ چه اتفاقی می‌افتد اگر مدخل‌های بیشتری داشته باشید؟ در مورد مدخل‌های کمتر چطور؟

(ب) فرض کنید چند محک^{۱۲} روی سیستم اجرا می‌کنید و می‌بینید که سیستم بیش از 99% از ظرفیت ورودی/خروجی صفحه‌بندی دیسک^{۱۳} خود را استفاده می‌نماید. اما تنها 10% از CPU استفاده می‌شود. چه ترکیبی از فضای دیسک و اندازه حافظه می‌تواند باعث این اتفاق شود؟ فرض کنید تعداد مدخل‌های TLB برابر با پاسخ قسمت قبل است.

(ج) از میان افزایش اندازه TLB، افزودن فضای دیسک بیشتر و افزودن حافظه‌ی بیشتر، کدام یک منجر به بزرگترین افزایش عملکرد خواهد شد؟ چرا؟

¹⁰Resident Set Size

¹¹Working Set Size

¹²Benchmark

¹³Paging Disk IO

تمارین عملی

۱. XV6 OS

برای این تمرین از کد بارگذاری شده در سامانه CW استفاده کنید.

در سیستم عامل xv6، فراخوانی سیستمی `fork()` تمام فضای حافظه کاربر پردازش والد را به طور کامل در پردازش فرزند کپی می‌کند. اگر پردازش والد بزرگ باشد، این کپی برداری زمان زیادی می‌برد. علاوه بر این، اغلب این کپی‌ها باعث هدررفت حافظه می‌شوند؛ زیرا در بسیاری از موارد، هیچ‌کدام از والد یا فرزند تغییری در یک صفحه خاص ایجاد نمی‌کنند و بنابراین می‌توانستند از همان حافظه فیزیکی مشترک استفاده کنند. این ناکارآمدی به‌ویژه زمانی برجسته‌تر می‌شود که پردازش فرزند بلافاصله پس از `fork()`، فراخوانی `exec()` را انجام دهد؛ زیرا `exec()` تمام صفحات کپی شده را دور می‌ریزد، بدون اینکه احتمالاً بیشتر آن‌ها حتی استفاده شده باشند. از سوی دیگر، اگر والد و فرزند هر دو از یک صفحه استفاده کنند و یکی (یا هر دو) روی آن بنویسند، در این حالت واقعاً نیاز به یک کپی مستقل وجود دارد. راه‌حل پیشنهادی برای این مشکل، استفاده از مکانیسم Copy-on-Write است.

هدف از پیاده‌سازی Copy-on-Write fork (یا COW fork)، به تعویق انداختن تخصیص و کپی صفحات حافظه فیزیکی برای پردازش فرزند تا زمانی است که واقعاً لازم شود (و اگر اصلاً لازم شود). نحوه عملکرد COW به شرح زیر است:

۱. تنها یک جدول صفحات^{۱۴} جدید برای پردازش فرزند ایجاد می‌شود.
 ۲. ورودی‌های جدول صفحه (PTE) مربوط به حافظه کاربر، به همان صفحات فیزیکی والد اشاره می‌کنند.
 ۳. تمام PTE‌های مربوط به حافظه کاربر، در هر دو جدول صفحات والد و فرزند، به‌عنوان غیرقابل نوشتن (با پاک کردن بیت `PTE_W`) علامت‌گذاری می‌شوند.
- هرگاه یکی از پردازش‌ها (والد یا فرزند) تلاش کند روی یکی از صفحات COW بنویسد، پردازنده یک خطای صفحه^{۱۵} تولید می‌کند. در این حالت، کنترل به `handler` خطای صفحه در هسته منتقل می‌شود. هسته تشخیص می‌دهد که این خطا مربوط به یک صفحه COW است و مراحل زیر را انجام می‌دهد:

- تخصیص یک صفحه فیزیکی جدید با استفاده از `kalloc()`
- کپی محتوای صفحه اصلی به صفحه جدید
- به‌روزرسانی ورودی جدول صفحه پردازش خطاکننده به گونه‌ای که:
 - به صفحه جدید اشاره کند
 - بیت `PTE_W` فعال شود

پس از بازگشت از `handler` خطای صفحه، پردازش می‌تواند بدون مشکل روی نسخه اختصاصی خود از صفحه بنویسد. در روش COW، آزادسازی صفحات فیزیکی پیچیده‌تر از حالت عادی است، زیرا یک صفحه فیزیکی ممکن است همزمان توسط چندین پردازش مورد ارجاع قرار گیرد. بنابراین، یک صفحه فیزیکی تنها زمانی باید آزاد شود که آخرین ارجاع به آن حذف شده باشد. برای مدیریت این مسئله، می‌توان از شمارنده ارجاع^{۱۶} در پیاده‌سازی تخصیص حافظه در فایل `kalloc.c` استفاده کرد. یک مسیر منطقی برای پیاده‌سازی Copy-on-Write به شرح زیر است:

۱. اصلاح تابع `uvmcopy()` به گونه‌ای که به جای تخصیص صفحات جدید، صفحات فیزیکی والد را در جدول صفحات فرزند نگاشت کند و بیت `PTE_W` را در هر دو پاک کند.
۲. تغییر تابع `usertrap()` برای تشخیص خطاهای صفحه ناشی از نوشتن روی صفحات COW و رسیدگی به آن‌ها.
۳. پیاده‌سازی شمارنده ارجاع برای صفحات فیزیکی به منظور آزادسازی صحیح حافظه.

¹⁴Page Table

¹⁵Page Fault

¹⁶Reference Count

۴. اصلاح تابع `copyout()` تا در مواجهه با صفحات COW، همان رفتار handler خطای صفحه را اعمال کند.

برای علامت‌گذاری نگاشت‌های COW، می‌توان از بیت‌های رزرو شده برای نرم‌افزار^{۱۷} در ورودی‌های جدول صفحه معماری RISC-V استفاده کرد. ماکروها و تعاریف مرتبط با این بیت‌ها در فایل `kernel/riscv.h` موجود هستند. برای بررسی صحت پیاده‌سازی، برنامه‌ای به نام `cowtest` در `xv6` ارائه شده است. این برنامه چندین سناریوی مختلف را آزمایش می‌کند. در `xv6` بدون پشتیبانی از COW، حتی اولین آزمون شکست می‌خورد، زیرا حافظه فیزیکی کافی برای کپی کامل پردازنده والد وجود ندارد. همچنین برنامه `usertests` نیز ارائه شده است. پس از پیاده‌سازی صحیح `COW fork`، انتظار می‌رود هر دو برنامه `cowtest` و `usertests` بدون خطا اجرا شوند. توجه کنید که تنها در صورتی که هر دو تست به‌طور کامل پاس شوند، نمره کامل دریافت خواهید کرد. برای اجرای این دو برنامه، می‌توانید پس از اجرا شدن سیستم، برنامه‌های کاربر آن‌ها را اجرا کنید.

در نهایت، کدهای تغییر یافته خود را در سامانه CW بارگذاری کنید.

۲. Docker

در این تمرین تمرکز ما بر روی مفاهیم مربوط به `OverlayFS` است تا با آشنایی و بررسی آن، بتوانیم در تمرین بعدی از آن برای پیاده‌سازی لایه‌های ایمیج استفاده کنیم.

نیازمندی به یک فایل سیستم قابل خواندن و نوشتن که براساس یک فایل سیستم موجود فقط خواندنی، پیشینه قدیمی‌تری از داکر و ابزارهای اجرای کانیتینر دارد. با این حال، استفاده از چنین فایل سیستمی برای پیاده‌سازی لایه‌های ایمیج‌ها و ایجاد سازوکارهای کش کردن، گزینه مناسبی است. برای اطلاعات بیشتر می‌توانید این مستند را بررسی کنید.

گام‌های تمرین

تمام دستورات را در مسیر `/tmp/overlay-lab` اجرا کنید.

آ. ابتدا چهار پوشه زیر را بسازید و فایل مربوطه را در پوشه `lower` تشکیل دهید.

```
1 mkdir lower upper work merged
2 echo "Content of file 1" > lower/file.txt
```

ب. به واسطه دستور زیر پوشه `upper` را بر روی پوشه `lower` به مقصد `merged` ماونت کنید.

```
1 sudo mount -t overlay overlay -o lowerdir=$(pwd)/lower,upperdir=$(pwd)/
  upper,workdir=$(pwd)/work $(pwd)/merged
```

ج. محتویات پوشه `merged` را بررسی کنید.

سوال: عدد inode فایل `file.txt` را در پوشه `merged` و `lower` مقایسه کنید. با تحقیق روی نحوه عملکرد `OverlayFS` نتیجه مقایسه را توجیه کنید.

د. حال یه فایل جدید به اسم `newfile.txt` داخل پوشه `merged` بسازید. بررسی کنید که این فایل علاوه بر این پوشه در کدام یک از پوشه‌ها ساخته شده است.

سوال: محتویات پوشه `work` را بررسی کنید و نقش آن در ایجاد `OverlayFS` را توضیح دهید.

ه. حال محتویات فایل `file.txt` را داخل پوشه `merged` تغییر دهید.

سوال: مقدار inode و محتویات فایل `file.txt` را در سه پوشه `lower`، `upper` و `merged` را مقایسه و تحلیل کنید.

¹⁷Reserved for Software (RSW)

و. پیاده‌سازی ایمپچ‌های داکر به صورت لایه‌لایه و OverlayFS صورت می‌گیرد. به این صورت که هر کانتینر از تعدادی پوشه lower و یک پوشه upper تشکیل می‌شود. ابتدا فایل سیستم قبلی را حذف کنید.

```
1 sudo umount merged
2 sudo rm -rf ./*
```

حال یک OverlayFS با چند لایه lower ایجاد کنید.

```
1 mkdir lower0 lower1 lower2 upper work merged
2 echo "Content of file 0" > lower0/file.txt
3 echo "Content of file 1" > lower1/file.txt
4 echo "Content of file 2" > lower2/file.txt
5
6 sudo mount -t overlay overlay -o lowerdir=$(pwd)/lower2:$(pwd)/lower1:$(pwd)
   )/lower0,upperdir=$(pwd)/upper,workdir=$(pwd)/work $(pwd)/merged
```

سوال: محتوا و مقدار inode فایل file.txt را در سه پوشه lower0 و lower1 و lower2 و همچنین merged را بررسی کنید و نحوه اولویت‌بندی در OverlayFS را توضیح دهید.

ز. حال می‌خواهیم یک نمونه ساده‌سازی شده از یک Dockerfile با فرمت زیر را به صورت لایه‌لایه build کنیم.

```
1 BASEDIR /tmp/overlay-lab/lower0
2
3 RUN echo "BIB1" > file1.txt
4 RUN echo "BIB2" > file1.txt
5 RUN rm file1.txt
6 RUN echo "BIB3" > file3.txt
```

شما باید یک Bash Script توسعه دهید تا ابتدا یک OverlayFS بر اساس BASEDIR ایجاد کند. سپس اولین RUN را در داخل پوشه merged0 اجرا کند. سپس فایل سیستم ایجاد شده را unmount کنید.

حال در مرحله بعدی، یک فایل سیستم OverlayFS جدید ایجاد کنید، که دو لایه lower آن به ترتیب lower0 و upper0 است و فایل سیستم در merged1 قرار می‌گیرد. دوباره با اجرای RUN دوم در پوشه merged1، فایل سیستم را unmount کنید.

در نهایت فایل سیستم نهایی ایجاد شده باید لایه‌های lower به ترتیب، lower0، upper0، upper1، upper2 و upper3 داشته باشد.

شما لازم نیست در توسعه این اسکریپت موارد مربوط به نام‌فضا و chroot را در نظر بگیرید و فقط مسائل مربوط به فایل سیستم مدنظر است. همچنین اسکریپت شما باید بتواند به صورت دینامیک از Dockerfile های مشابه پشتیبانی کند.

موارد لازم جهت تحویل

- پاسخ به چهار سوال.
- اسکریپت توسعه داده شده در بخش ز.
- توضیحات کامل از نتایج حاصل از اجرا اسکریپت، بررسی پوشه‌ها، فایل‌های ساخته شده و فایل سیستم‌های ایجاد شده.

۳. توسعه سیستم مدیریت پرونده

این تمرین در ادامه تمرین قبل طراحی شده است و قصد داریم سامانه مدیریت پرونده خود را تکمیل کنیم. در این تمرین می‌خواهیم یک الگوریتم سریعتر و کم مصرف‌تر برای مدیریت فضاها، خالی پیاده‌سازی کنیم. تا اینجای کار داده ساختاری که برای اینکار در نظر می‌گرفتیم یک لیست پیوندی بود اما همانطور که می‌دانید لیست‌های پیوند داده ساختارهای چندان مطلوبی نیستند. در این تمرین یک داده ساختار مبتنی بر bitmask را برای اینکار پیاده می‌کنیم:

گام‌های تمرین

۱. یک دستور با نام stressTest تعریف کنید به طوری که با زدن این تابع یک فایل سیستم جدید بسازد سپس روی آن ۱۰۰۰۰ فایل ایجاد کند سپس ۱۰۰۰۰۰۰ بار روی این فایل سیستم جدید عملیات‌های مقابل را انجام دهد: از فایل‌های موجود به تصادف بخواند - بنویسد طول فایل‌ها را تغییر دهد - فایل جدید بسازد - یک فایل را پاک کند. شما مجاز هستید در صورتی که مدت زمان اجرای این تابع از ۱ دقیقه بیشتر شد پارامترهای آن را تغییر دهید.
 ۲. با استفاده از ابزارهایی که در تمرینات گذشته با آن‌ها آشنا شدید (مانند tracy و perf) در مورد وضعیت الگوریتم‌های فایل سیستم اطلاعات بدست آورید و گلوگاه‌های احتمالی آن را بیابید.
 ۳. برای فایل سیستم اندازه بلوک را برابر با ۴KB در نظر بگیرید در این صورت ما در این فایل سیستم ۳۲K بلوک داریم. حال وضعیت آزاد یا گرفته شده بودن هر یک از این بلوک‌ها را با استفاده از ۱ بیت می‌توانیم نمایش دهیم در این صورت به ۳۲Kb نیازمندیم که این مقدار برابر با ۴KB است. در این صورت ما کفایت صرفاً با استفاده از یکی از این بلوک‌ها وضعیت گرفته یا آزاد بودن هر یک از آن‌ها را نمایش دهیم.
 ۴. بلوک ۰ را برای metadata می‌گذاریم و بلوک ۱ را برای freelist جدیدمان. حال توابع alloc و free را به شکل مناسب تغییر دهید.
 ۵. کارایی الگوریتم جدید خود را اندازه‌گیری نمایید و با حالت گذشته آن مقایسه کنید.
- توجه کنید که در گزارش‌های مربوط به کارایی باید flamegraph یا نمودار tracy را بدست آورید و نقاط قابل توجه آن را مورد ارزیابی قرار دهید.